Milestone-1

INVENTRA–INTELLIGENT INVENTORY MANAGEMENT SYSTEM

MODULE 1: AUTHENTICATION & SECURITY MODULE

- DEVELOPED USING: JAVA 17+, SPRING BOOT, SPRING SECURITY
- DATABASE: MYSQL
- FRONTEND: HTML, CSS, BOOTSTRAP, THYMELEAF

Presented by: Arya Desavale

Problem Statement

Businesses need secure access control in inventory systems.

Without authentication:

- Unauthorized access may occur
- Data manipulation risk increases
- No user accountability
- No role-based restrictions

There is a need for:

- Secure login system
- Role-based access (Admin, Manager, Staff)
- Password encryption
- Token-based authentication

EXISTING PROBLEM

Traditional inventory systems:

- Store passwords in plain text
- ❌ No role separation
- ❌ No JWT token authentication
- ❌ Weak password reset mechanism
- ❌ No proper session security

These systems are vulnerable to:

- Data breaches
- Unauthorized access
- Session hijacking

PROPOSED SYSTEM:

Inventra Authentication Module provides:

- ✅ Secure signup (Admin-controlled)
- ✅ Encrypted password storage
- ✅ JWT-based authentication
- ✅ Role-based access control
- ✅ Secure forgot password system
- ✅ Token-based password reset

Security implemented using:

- Spring Security
- JWT Utility
- Encrypted password hashing
- Email reset token mechanism

PROPOSED SYSTEM:

Inventra Authentication Module provides:

Module	Purpose
User Registration	Create new user with encrypted password
Login	Authenticate and generate JWT token
Email Verification	Verify user identity
Forgot Password	Send reset link via email
Reset Password	Update password securely
Dashboard Access	Protected access using JWT

ARCHITECTURE (DB + FLOW)

Signup:
UI → Controller → Service → Repository → Database

Signin:
UI → Controller → Service → JWT Token → UI

Forgot Password:
UI → Controller → Service → Email Service → User

Database Table: USER

Fields:

id (Primary Key)
username (Unique)
password (Encrypted)
email
role (Admin / Manager / Staff)
reset_token
token_expiry

Purpose:
To secure the system by verifying user identity.

Functionalities:

- Signup
- Login
- Forgot Password
- Reset Password
- JWT Token Generation

CLASSES IN MODULE

Methodology

AuthController

Handles HTTP requests

AuthService

Contains business logic

User

Entity class (mapped to DB)

UserRepository

Interface to interact with database

JWTUtility

Generates and validates tokens

EmailService

Sends reset password email

Pseudocode for Each Class

AuthController:

```
CLASS AuthController

FUNCTION signup(request)
   result = AuthService.registerUser(request)
   RETURN result
END FUNCTION

FUNCTION signin(request)
   token = AuthService.authenticate(username,
password)
   RETURN token
END FUNCTION

FUNCTION forgotPassword(email)
   result = AuthService.processForgotPassword(email)
   RETURN result
END FUNCTION

END CLASS
```

AuthService – Register

```
FUNCTION registerUser(userDetails)

   existingUser =
UserRepository.findByUsername(username)

   IF exists THEN
      RETURN "User already exists"
   END IF

   encryptedPassword = encrypt(password)

   create new user
   save user

   RETURN "Signup successful"

END FUNCTION
```

Reset Password

```
FUNCTION resetPassword(token,
newPassword)

   user = findByResetToken(token)

   IF invalid
      RETURN "Invalid token"
   END IF

   encrypt newPassword
   update password
   clear token

   RETURN "Password reset successful"

END FUNCTION
```

Pseudocode for Each Class

AuthService – Login

```
FUNCTION authenticate(username, password)

    user = findByUsername(username)

    IF user not found
        RETURN "Invalid username"
    END IF

    IF password matches
        token = JWTUtility.generateToken(user)
        RETURN token
    ELSE
        RETURN "Invalid password"
    END IF

END FUNCTION
```

Forgot Password

```
FUNCTION processForgotPassword(email)

    user = findByEmail(email)

    IF not exists
        RETURN "Email not registered"
    END IF

    generate resetToken
    save token

    EmailService.sendResetLink(email, token)

    RETURN "Reset link sent"

END FUNCTION
```

Conclusion – Module 1

The Authentication Module:

- Secures access to the system
- Prevents unauthorized usage
- Encrypts sensitive data
- Uses JWT for stateless authentication
- Implements secure password reset
- This module forms the security foundation of Inventra.

References

- Java 17 Documentation
- Spring Boot Official Documentation
- Spring Security Documentation
- JWT Documentation
- MySQL Official Docs
- Hibernate / JPA Docs

The End